# The Addition Matrix as Rotors

**A counting law for digit-by-digit factor search, and the address of the final answer: a code paper, with computed checks**

**Cody Michael Allison** (Michael Rendier), independent researcher · the.wandering.god@gmail.com · ORCID 0009-0007-7239-6760 · github.com/michaelrendier
**Written by an AI assistant:** Claude Sonnet 5.5 (`claude-sonnet-5-5`), with the author's `cs-paper-code-conventions` skill running
**Date:** 14 October 2026 · **Status:** stub for expert review · **Code:** FourthAgePapers/Addition-MatrixOrbits

---

## Read this first: who wrote this, and how

**The assistant.** This paper was written by a transformer language model, Claude Sonnet 5.5 (`claude-sonnet-5-5`), working with the author's `cs-paper-code-conventions` skill active; the skill decides how code is shown in a paper (linked above and described in Appendix A). The code and the text are from 2026-10-14.

**The author.** The author is a software engineer, not a trained mathematician, and says so. He is asking readers who are mathematicians to read this critically. **Every statement here that is not attributed to a cited source should be treated as unverified until an expert has looked at it.**

**The method.** The author deliberately does not tell the assistant what result he is looking for or hoping for. Tasks are given as measurements ("find the law", "is the answer at an address"), so that he is not handed the answers he wants. This is a safeguard, not a guarantee: an assistant can still infer an expectation from context. The real safeguards are exact enumeration, a control before each measurement, and the rule that **failures stay in the code and in the paper** (Section 7).

**What kind of paper this is.** A code paper: it shows the programs, what they compute, how to run them and what came out. It contains no proofs and **no prediction**. Where a result is in the literature it says so and cites it.

---

## Abstract

Grade-school multiplication writes one row per digit of the multiplier; adding the rows column by column gives the product. We read that addition matrix as arithmetic modulo $10^k$ and report, with a small standard-library program, what it computes. (1) The matrix, with carries in {0, 1}, reproduces the product on 203 cases (failures: 0). (2) The digit in column *c* is a wheel in the multiplier with an exact period 10^(c+1)/gcd(num1, 10^(c+1)); the cycle type of $x \to a \cdot x + b$ on $\mathbb{Z}/10^m$ equals the Chinese-remainder combination of its types on $\mathbb{Z}/2^m$ and $\mathbb{Z}/5^m$ (18040 maps, 0 failures); the odometer $x \to x + 1$ returns, on the multiples of 10, to the odometer one scale down. (3) Digit-by-digit factor reconstruction keeps exactly the pairs (p, q) mod 10^(c+1) with $p \cdot q \equiv N$ there; their number has a closed form, a product over the primes 2 and 5, equal to $(v_2+1)(v_5+1) \cdot 4 \cdot 10^c$ when both valuations are below c+1: unit-group torsors, ×10 per column (closed form against enumeration, 2370 cases, 0 mismatches). (4) The overflow $K = (p'q' - N)/10^n$ of a unit solution is an integer and the true factorisation is exactly K = 0: it has an address. The address is not isolated, because near-misses with small |K| exist. Nothing here is a factoring method; the live-state count is the cost of the search it describes.

## 0. Scope, labels, and what this paper is not

- **Not a Fourth Age paper.** It has no pre-registered prediction. It is a code paper with results.
- **Labels.** `ESTABLISHED` (textbook or published), `OURS` (not found in the literature we read), `THEORETICAL`, `OPEN`. The suffix `:CALCULATED` means this paper's own code produces the result; it is not a derivation. Everything in Sections 1–4 is `ESTABLISHED:CALCULATED` (elementary number theory, checked by the code over the stated range) except the address of Section 4, which is `OURS:CALCULATED` (a restatement; reported with its neighbours).
- **Built-against and Post-Hoc.** The Multiplication Matrix Explorer, whose matrix and factor search this paper analyses, is the author's own earlier tool (built-against, no tag). Heninger–Shacham (2009) was noticed afterwards and is marked `Post-Hoc:` in the references. We have not been able to download or read it.
- **The generational-lineage engine is not used here.** The engine files in `engine/` are plain Python; no section depends on it.
- **Not claimed.** No factoring algorithm, no speed-up, no claim about any cryptosystem.

## 1. The addition matrix

For `num1 × num2` with `num2 = Σ dⱼ 10ʲ`, row *j* is the decimal digits of `num1·dⱼ` starting at column *j*. Adding the rows one at a time into a running total keeps every carry in {0, 1}. `carry_matrix` in `engine/e_addm_orbits.py` does this on the Explorer's 21-column grid (`N_COLUMNS = 21`). It was checked against Python's `*` on 203 cases including 0, the largest inputs and 200 random pairs: 0 failures; the largest carry seen over the first 60 cases is 1. **[ESTABLISHED:CALCULATED]**

## 2. The columns as rotors

**Column period.** Column *c* of `num1 × num2` depends on `num2` only modulo 10^(c+1), and its digit has exact period 10^(c+1)/gcd(num1, 10^(c+1)) in `num2`. The formula and a divisor search over a full range agree on all 30 (num1, c) pairs tried (disagreements: 0). **[ESTABLISHED:CALCULATED]**

**Two stacks of wheels.** The ring $\mathbb{Z}/10^m$ is $\mathbb{Z}/2^m \times \mathbb{Z}/5^m$. For a permutation acting on coprime factors, a cycle of length $l_1$ and one of length $l_2$ give $\gcd(l_1, l_2)$ cycles of length $\mathrm{lcm}(l_1, l_2)$. The cycle type of $x \to a{\cdot}x + b$ on $\mathbb{Z}/10^m$ equals this combination for every unit a and a spread of b at m = 1, 2, 3: 18040 maps, 0 failures. Example, $x \to 3x + 1$: on $\mathbb{Z}/1000$ the cycle type is {100: 8, 20: 8, 4: 10}, on $\mathbb{Z}/8$ it is {4: 2} and on $\mathbb{Z}/125$ it is {100: 1, 20: 1, 4: 1, 1: 1}. **[ESTABLISHED:CALCULATED]** The wiring `num1 × digit` is a bijection of $\mathbb{Z}/10^m$ only when `num1` is coprime to 10; for other `num1`, `affine_cycle_type` raises `ValueError` rather than report a cycle type that does not exist (checked: True).

**The odometer.** `x → x + 1` on $\mathbb{Z}/10^m$ is one cycle of length $10^m$. Its first return to the multiples of 10 is `x → x + 10`, which after dividing by 10 is the odometer on $\mathbb{Z}/10^{(m-1)}$: True for m = 2, 3, 4. This is the adding machine on the 10-adic integers, self-similar under a change of scale by 10. **[ESTABLISHED:CALCULATED]** On $\mathbb{Z}/2^n$, every cycle length of every $x \to a{\cdot}x + b$ with a odd (a in 1, 3, 5, 7, 11; b in 0, 1, 3; n = 8) is a power of two: True.

## 3. The law of the factor search

Digit-by-digit factor reconstruction, from the units end, extends both factors by one digit per column and keeps a pair only if the column's digit of p·q equals N's. Listing 1 is the search; it returns the live-state count after each column.

**Listing 1 — live states of the digit-by-digit factor search (`engine/e_addm_orbits.py`, `live_state_counts`; verbatim, executed cold)**

```python
from typing import List


def live_state_counts(N: int, n_digits: int) -> List[int]:
    states, counts = [(0, 0)], []
    for c in range(2 * n_digits):
        mod, target, nxt = 10 ** (c + 1), N % 10 ** (c + 1), []
        if c < n_digits:
            step = 10 ** c
            for p, q in states:
                for dp in range(10):
                    P = p + dp * step
                    for dq in range(10):
                        Q = q + dq * step
                        if (P * Q) % mod == target:
                            nxt.append((P, Q))
        else:
            nxt = [(p, q) for p, q in states if (p * q) % mod == target]
        states = nxt
        counts.append(len(states))
    return counts
```

```
>>> live_state_counts(14351, 3)
[4, 40, 400, 48, 4, 2]
```

After column *c* (for c below the digit count n) the live pairs are exactly the pairs (p, q) mod 10^(c+1) with p·q ≡ N there, so their number is the number of solutions of one congruence. By the Chinese remainder theorem it is a product over the primes 2 and 5, and Listing 2 gives it in closed form: with v = v_p(N) capped at k, f_p = (v+1)·φ($p^k$) when v < k and f_p = k·φ($p^k$) + $p^k$ when N ≡ 0 mod $p^k$.

### Listing 2 — the closed form (`engine/e_addm_orbits.py`; verbatim, executed cold)

```python
from typing import Optional


def valuation(N: int, p: int, cap: Optional[int] = None) -> int:
    if N == 0:
        return cap if cap is not None else 10 ** 9
    v = 0
    while N % p == 0 and (cap is None or v < cap):
        N //= p
        v += 1
    return v


def phi_prime_power(p: int, k: int) -> int:
    return p ** (k - 1) * (p - 1)


def prime_power_solutions(N: int, p: int, k: int) -> int:
    v = valuation(N, p, cap=k)
    return (v + 1) * phi_prime_power(p, k) if v < k else k * phi_prime_power(p, k) + p ** k


def solution_count(N: int, k: int) -> int:
```

```
    return prime_power_solutions(N, 2, k) * prime_power_solutions(N, 5, k)

>>> [solution_count(14351, c + 1) for c in range(3)]
[4, 40, 400]
>>> solution_count(420, 2), solution_count(420, 1)
320 27
```

| N | $v_2$ | $v_5$ | live states per column | closed form, columns below n |
|---|---|---|---|---|
| 3233 | 0 | 0 | [4, 40, 6, 2] | [4, 40] |
| 14351 | 0 | 0 | [4, 40, 400, 48, 4, 2] | [4, 40, 400] |
| 1022117 | 0 | 0 | [4, 40, 400, 4000, 406, 38, 8, 2] | [4, 40, 400, 4000] |
| 420 | 2 | 1 | [27, 320, 32, 16] | [27, 320] |
| 850 | 1 | 2 | [27, 260, 26, 6] | [27, 260] |
| 5100 | 2 | 2 | [27, 520, 32, 4] | [27, 520] |

The closed form was also compared with enumeration (for each first factor $a$, the equation $a \cdot b \equiv N$ has $\gcd(a, 10^k)$ solutions when that divides N) for every N in $[0, 2 \cdot 10^k + 50)$, k = 1, 2, 3: 2370 cases, 0 mismatches. **[ESTABLISHED:CALCULATED]**

For N coprime to 10 the count at column c is $4 \cdot 10^c$, the order of the unit group $(\mathbb{Z}/10\text{^}(c+1))^\times$ = [4, 40, 400, 4000] for k = 1..4. The solution set is a torsor of that group acting by $(p, q) \to (u \cdot p, u^{-1} \cdot q)$; with 2s and 5s in N it is $(v_2+1)(v_5+1)$ torsors. Every unit solution has exactly ten lifts to the next column (children per solution for N = 14351 at k = 2 and N = 3233 at k = 1: {10: 40} and {10: 4}). The "×10 per added digit" growth the Explorer measured is therefore the order of a unit group. From column n on, the digits of the factors are no longer free and the count collapses (to 2: the true pair and its swap). **[ESTABLISHED:CALCULATED]**

## 4. The address of the final answer

For a unit solution $(p', q')$ mod $10^n$, the number $K = (p'q' - N)/10^n$ is an integer. If $p', q' < 10^n$ are the true factors, K = 0, and K = 0 for a solution exactly when $N = p' \cdot q'$. Listing 3 enumerates the solutions with their overflow.

**Listing 3 — the unit solutions mod $10^n$ and their overflow K (`engine/e_addm_orbits.py`; verbatim, executed cold)**

```
from math import gcd
from typing import List, Tuple


def unit_solutions(N: int, n: int) -> List[Tuple[int, int, int]]:
    M = 10 ** n
    if gcd(N, 10) != 1:
        raise ValueError("unit_solutions needs N coprime to 10")
    out = []
    for p in range(1, M):
        if gcd(p, 10) != 1:
            continue
        q = (N * pow(p, -1, M)) % M
        out.append((p, q, (p * q - N) // M))
    return out
```

```
>>> sols = unit_solutions(101 * 103, 3); len(sols)
400
>>> [s for s in sols if s[2] == 0]
[(101, 103, 0), (103, 101, 0)]
>>> sorted(abs(s[2]) for s in sols if s[2])[:5]
[1, 1, 1, 1, 2]
```

On 30 products of two 3-digit primes (n = 3), every N had 400 unit solutions and 2 with K = 0 (the true pair and its swap). Among the other solutions the smallest nonzero |K| ranged from 1 to 15; the median of the per-N median |K| was 164. **The address exists and is unique among zero-overflow solutions, but it is not isolated:** near-misses with |K| of 1, 2, … exist, and no coordinate of the unit group distinguishes the true pair in advance. Testing K needs the whole solution set, which has 4·10^(n−1) elements. **[OURS:CALCULATED]**

**What partial knowledge does.** With the low k digits of p known (N = 1009·1013, p = 1009), the number of survivors by column is: k = 1: [1, 10, 100, 1000]; k = 2: [·, 1, 10, 100]; k = 3: [·, ·, 1, 10] (· = before the known digits end). Each known low-order digit divides the live set by 10, and the first also removes the factor 4. This is the lift that branch-and-prune key-recovery methods use when part of a key is known (Heninger and Shacham 2009, `Post-Hoc:`, not read by us); it gives no advantage when nothing is known. **[ESTABLISHED:CALCULATED]** for the counts.

## 5. Results, in one place

The matrix reproduces the product; the columns are wheels with exact periods; $\mathbb{Z}/10^k$ is two stacks of wheels, the odometer is self-similar; the live-state count of the factor search is the closed form of Section 3, the order of a unit group, ×10 per column; the true factorisation is the K = 0 address of Section 4 and is not isolated. It does **not** factor anything faster than the search it counts, and it does not bear on any cryptosystem.

## 6. Follow-up tests

Three questions that the first results raised, each answered by a small program in `engine/e_addm_followup.py` (the same code as the Explorer's `toolsets/torsor.py`, `treeaut.py`, `closedform.py`).

**6.1 Is there a closed form when N is divisible by $2^k$ or $5^k$, and is it right?** With v = v_p(N) capped at k: $a \cdot b \equiv N \pmod{p^k}$ has $(v+1)\cdot\varphi(p^k)$ solutions when $v < k$ and $k\cdot\varphi(p^k) + p^k$ when $N \equiv 0 \pmod{p^k}$. The derivation (split a by its valuation; each valuation $i \le \min(v, k-1)$ contributes $\varphi(p^k)$ pairs; a = 0 contributes $p^k$ more when $N \equiv 0$) is in the toolset's docstring; the code is the check. Enumerating every N modulo $p^k$ against every a, there were no mismatches for p = 2 with k = 1..10, p = 3 with k = 1..6, p = 5 with k = 1..4, p = 7 with k = 1..3 and p = 11 with k = 1..2 (25 prime powers, 0 with any mismatch). Modulo $10^k$ the count is the product over p = 2, 5; every N for $k \le 3$ and ten selected N (including 0, $2^k$, $5^k$ and $10^k/2$) for k = 4, 5 agree: no mismatches. **[ESTABLISHED:CALCULATED]**

**6.2 Why are the cycle lengths on $\mathbb{Z}/2^n$ powers of two?** The residues mod $2^k$ form a binary tree of depth n. Its automorphism group has order $2^{(2^n-1)}$ (a swap of the two children at each of the $2^n - 1$ internal nodes, independently): counted by brute force over every permutation of the leaves for n = 1, 2, 3 ({1: 2, 2: 8, 3: 128}, equal to $2^{(2^n-1)}$: True), and by the recursion $|\mathrm{Aut}(T_n)| = 2\cdot|\mathrm{Aut}(T_{n-1})|^2$ to 32768 for n = 1..4. It is a 2-group, so every element has power-of-two order and every cycle has a power-of-two length. The affine maps $x \to a\cdot x + b$ (a odd) preserve the tree (True) and form a group of order $2^{(2n-1)}$ (counted as distinct maps for n = 2..6: {2: (8, 8), 3: (32, 32), 4: (128, 128), 5: (512, 512), 6: (2048, 2048)}), so they lie in that 2-group; every cycle length of every one of them on $\mathbb{Z}/256$ is a power of two (True). On $\mathbb{Z}/5^m$ the unit group has order $4\cdot5^{(m-1)}$, and the cycle lengths of $x \to 2x + 1$ are

{1: [1, 4], 2: [1, 4, 20], 3: [1, 4, 20, 100], 4: [1, 4, 20, 100, 500]} for m = 1..4: not powers of 5. **[ESTABLISHED:CALCULATED]**

**6.3 Can the factors be found by walking down the solution tree?** For N coprime to 10 the unit solutions form a uniformly 10-ary tree (every node has ten children at k = 1 and k = 2 for N = 14351: {10: 4} and {10: 40}). The true factorisation is one root-to-leaf path, so a walk needs something at a node that prefers the true child among its ten siblings. We measured three candidate guides, each a rank of the true child among its siblings (0 = ranked first, about 0.5 = no information): the overflow |K| of Section 4; the size bound (the child with p′q′ closest to N from below, the exact fact that p′q′ ≤ N on the true path); and the smallest p′ (a control with no relation to the truth); plus a random child. Normalised mean rank and z-score against 0.5 (ranks are averaged over ties; n = digits per prime, level = child modulus 10^level):

| n | level | trials | overflow = size: mean rank (z) | smallest p′: mean rank (z) | random child: mean rank (z) |
|---|---|---|---|---|---|
| 3 | 2 | 200 | 0.516 (+0.8) | 0.494 (-0.3) | 0.543 (+2.1) |
| 3 | 3 | 200 | 0.001 (-24.5) | 0.519 (+0.9) | 0.543 (+2.1) |
| 4 | 2 | 200 | 0.531 (+1.5) | 0.484 (-0.8) | 0.529 (+1.4) |
| 4 | 3 | 200 | 0.522 (+1.1) | 0.488 (-0.6) | 0.529 (+1.4) |
| 4 | 4 | 100 | 0.000 (-17.3) | 0.498 (-0.1) | 0.530 (+1.0) |
| 5 | 3 | 100 | 0.482 (-0.6) | 0.518 (+0.6) | 0.522 (+0.8) |
| 5 | 4 | 100 | 0.551 (+1.8) | 0.459 (-1.4) | 0.522 (+0.8) |

Below the last level no guide departs from chance beyond the scatter of repeated runs: |z| is about 2 or less in every cell above, and the random-child control itself reads +2.1 in two of them. Two cells first looked borderline in exploratory runs and were chased with more trials and other seeds (`claude/scratchpad/2026-10-14_addition_matrix_followups/`): n = 4, level 3 read z = +1.9 at 60 trials and +0.2 and +1.5 at 400 trials; n = 5, level 4 read −2.0 at 100 trials and −1.3, −0.7 and +2.2 at 300 trials with three further seeds. Both signs occur, which is scatter. At the last level the overflow is 0 for the true child, so the rank is about 0: that is the K = 0 address of Section 4 and says nothing before the walk has reached it. The exact pruning p′q′ ≤ N removes nothing until the last level: for N = 14351, the solutions surviving at levels 1, 2, 3 are (4, 40, 26) of (4, 40, 400). The two guides “overflow” and “size” give identical rankings, because both are the distance from p′q′ to N. **[OURS:CALCULATED]** The answer to the question as posed is that no tested guide prefers the true child before the last level, and the size bound acts only at the last level; this is a measurement on the guides we tried, not a proof that none exists.

## 7. Faults and corrections, kept

| # | what | kind | status |
|---|---|---|---|
| 1 | The first cycle-type function was valid only for permutations; run over every affine map it gave false CRT failures for the non-unit multipliers (117 of them), and was corrected to raise on a non-permutation | CODE | fixed; the non-unit case now raises |
| 2 | The Explorer's `matrix_math.py` prose says "20 columns"; its constant `N_COLUMNS` is 21; the first version of this engine used 20 and failed the cell-for-cell comparison | CODE (documentation), in our first version | engine matches the constant; the Explorer docstring is for the author to reword |
| 3 | Two rank-test cells first read | z | near 2 for the overflow guide (n = 4, level 3: +1.9; n = 5, level 4: −2.0); chased with more trials and seeds, both signs appear |

## 8. Questions for the reader

1. **Section 3.** Is the closed form for N divisible by $2^k$ or $5^k$, $k \cdot \varphi(p^k) + p^k$, correct as stated? We derived it by hand and checked it by enumeration for $k \leq 3$ only.
2. **Section 2.** Is the first-return statement for the odometer, and the CRT combination rule for cycle types, standard enough to cite to a textbook? We did not search for the source.
3. **Section 4.** Is there a known name or result for the overflow K as an invariant of the solution set mod $10^n$? We did not find one; absence of a search is not absence of the result.
4. **Section 4.** Does the decimal-column count relate to the published branch-and-prune analyses beyond the pointer given? We have not read that paper.

## 9. Reproducibility

```
cd FourthAgePapers/AdditionMatrixOrbits
python3 engine/e_addm_orbits.py              # about 12 s: every number in Sections 1-5
python3 engine/e_addm_orbits.py --crosscheck # compares with the ValaQuenta engine addition_matrix v1.0 (needs it
python3 engine/e_addm_followup.py            # about 15 s: Section 6
python3 engine/e_addm_followup.py --crosscheck  # compares with the Explorer's toolsets (next to this repo, or EX
python3 paper/make_paper.py --check          # does this paper match a fresh build?
```

Environment of this build: Python 3.12.3, a ThinkPad X1 Carbon (6th gen, Core i7-8550U); building the paper took about 18 s. SHA-256 prefixes of the engine file used:

| file | sha256 (first 12) |
|---|---|
| engine/e_addm_orbits.py | 063001a6dfba |
| engine/e_addm_followup.py | e18f24e2db2c |

Related code, not needed to reproduce this paper: the ValaQuenta engine `addition_matrix` (v1.0, 11 tests, one of which compares every row, carry and live-state count with the Multiplication Matrix Explorer), and the Explorer's `toolsets/law.py`.

## References

**The Multiplication Matrix Explorer is built-against and carries no tag; every numbered item below is `Post-Hoc:`.** The standard facts used (the Chinese remainder theorem, the structure of the unit group of $\mathbb{Z}/p^k$, Euler's $\varphi$) are textbook material and are not cited to a particular source here.


1. `Post-Hoc:` N. Heninger, H. Shacham, Reconstructing RSA private keys from random key bits, in *CRYPTO 2009*, LNCS 5677; ePrint 2008/510. (not downloaded: the host returned a bot-challenge page, not bypassed; fetch by hand)
2. (built-against; the author's own software, no tag) C. M. Allison, Multiplication Matrix Explorer, https://github.com/michaelrendier/MultiplicationMatrixExplorer.
3. (companion paper by the same author and assistant, which met the odometer and the 2-adic maps) *A Bit Shift Wearing A Disguise*, FourthAgePapers/CollatzShift.


## Appendix A. What in ContextPlease was used, and what ContextPlease is

**What ContextPlease is.** ContextPlease is a versioned, human-readable archive of the working context of the author's projects, kept so that a model, or a person, who has never seen the work can come up to speed with historical access. It holds the standing rules and notation, a per-repository current-state file, a dated log of what happened, the *skills* (reusable working methods the assistant is given), every research scratchpad with its scripts and outputs, copies of context primers and wiki pages, and, since 2026-10-14, a ledger of every third-party work downloaded. It is deliberately "manual transmission": nothing in it is loaded automatically; the reader chooses what to read and in what order. A new model should start with `README.md` (rules, traps, layout), then `claude/.clauderc_context_1` (what is true now) and `claude/.clauderc_context_2` (what happened), then the skills it is pointed at.

**What was used for this paper.**

| in ContextPlease | role in this paper |
|---|---|
| `README.md` | the standing rules in force: all failures stay in the code and the data |
| `claude/skills/cs-paper-code-conventions/SKILL.md` | how code appears in a paper: verbatim, numbered, executed-cold listings for short mechanisms |
| `claude/scratchpad/2026-10-14_addition_matrix_orbits/` | the exploratory scripts and outputs (`orbits.py`, `law_and_address.py`) before they were folded into `engine/` |
| `claude/scratchpad/2026-10-14_addition_matrix_followups/` | the chase of the two borderline rank-test cells (Section 6.3) |
| `claude/scratchpad/2026-09-26_addition_matrix_sudoku/` | the earlier factor-search script and its measured ×10 growth |

| in ContextPlease | role in this paper |
|---|---|
| claude/references/INDEX.md | the literature ledger, including the work that could not be downloaded |
| claude/.clauderc_citations | the queue of citations by repository |
| claude/.clauderc_canonical_maths (block @RCCM_CONFIDENCE_TIERS) | the label tiers, including the :CALCULATED suffix |
| claude/hooks/clauderc_primer.py | the start-of-session map of the files above (documentation copy) |

Not in ContextPlease: the assistant's local memory notes (for example the rule that every download of another person's work is recorded, and that outbound requests carry a browser-style user agent). Their content is summarised in the ledger and the primer where it matters.

*This appendix is a standing section of every paper the author now writes with an assistant.*